Project Sprint #5

GitHub link: GitHub - zran28320-lgtm/CS449-Solitaire · GitHub

The main tasks of this assignment are:

(1) Conducting a code review exercise.

(2) Conduct review of your tests.

(3) Summarizing the lessons learned from Sprint 0 through Sprint 5.

Total points: 20

1. Code Review (8 points)

Apply source code review to one or two most important classes (and other classes if time permits) and report the findings. In addition to looking for bugs, the review should check: (1) whether the entire project has followed the coding standard in a consistent manner, (2) whether the project has followed the design principles introduced in class, and (3) whether there are code smells that indicate the need for refactoring. The following checklists provide basic guidelines. You may add new items to each of the checklists.

Make sure your answers resulted from the code review exercise. If there is no finding for an entry, you should provide an explanation. For example, if your answer to "Are the naming conventions violated?" is no, you should describe a naming convention and present an example. You will receive no credit for this part if your answers are simply yes or no without additional information.

Classes that have been reviewed: SolitaireGame, ManualGame, AutomatedGame, SolitaireGUI, TestManualGame, TestAutomatedGame

Date/time duration of the code review exercise: April 23, 2026, about 60 minutes

Checklist	Checklist Item	Findings
Coding Standards	Are the naming conventions violated?	No major naming convention violation was found. The code generally follows Python naming conventions: classes use CamelCase, such as SolitaireGame, ManualGame, AutomatedGame, and SolitaireGUI; methods and variables use snake_case, such as set_board_parameters, start_new_game, validate_move, and make_automated_move. Test method names also follow the test_... convention, such as test_ac_1_1_valid_board_size and test_recording_feature.
	Is the ordering convention of method arguments violated?	No serious violation was found. The movement-related methods use a consistent argument order: source row, source column, end row, end column, as

		shown in validate_move(sr, sc, er, ec) and execute_move(sr, sc, er, ec, is_replay=False). This order is also used consistently in the tests, for example execute_move(3, 1, 3, 3).
	Any comments meaningless or inconsistent with the code?	No obviously meaningless or misleading comments were found. The comments are limited and mostly useful, such as the comment in replay mode explaining that replay should not be recorded again. The test file also includes a useful comment explaining that a new game is re-initialized before testing recording. However, the production code has very few comments explaining important design choices, such as why grid values use 0, 1, and 2. Adding comments or constants for these values would improve maintainability.
	Any code block has an inconsistent formatting style?	Some formatting issues were found, especially in gui_app.py. Several Tkinter widget creation statements are very long and split across multiple lines in a way that is hard to read, especially the radio button and button setup code in _setup_ui(). The logic file is generally cleaner, but validate_move() contains a long conditional expression for checking horizontal, vertical, and diagonal moves. These blocks could be reformatted or extracted into helper methods to better follow a consistent style.
	Any indentations inconsistent?	No indentation inconsistency was found. The code uses consistent indentation across the logic, GUI, and test files. Class definitions, method definitions, loops, and conditional blocks are properly indented in the reviewed codes.
Design Principles	Any class/method not well-modularized?	Yes. The project is modularized well at a high level because the game logic is separated from the Tkinter GUI, and the logic uses a class hierarchy with SolitaireGame as the base class and ManualGame / AutomatedGame as

		subclasses. This matches the project requirement to separate user interface and game logic and to use class hierarchy for manual and automated games. However, some methods are still not well modularized. For example, SolitaireGUI.on_replay_click() handles file selection, file parsing, game object creation, UI updating, replay animation, JSON parsing, and error handling all in one method. This method should be split into smaller methods such as load_record_file(), parse_init_line(), apply_replay_action(), and animate_replay().
	Any class with poor abstraction?	Yes. The board is represented with raw integer values: 0, 1, and 2, where these appear to mean invalid slot, peg, and empty hole. This works functionally, but the abstraction is weak because the meaning of each number is not obvious. Constants such as INVALID_SLOT = 0, PEG = 1, and EMPTY = 2 would make the code easier to understand. The board type is also represented as raw strings such as "English", "Diamond", and "Hexagon". This could be improved by using constants or separate board-shape classes.
	Is the visibility of any variable, method, and class inappropriate?	Yes. The code correctly uses leading underscores for helper methods such as _is_valid_slot() and _generate_board(), which signals that they are internal implementation details. However, important internal state variables such as grid, pegs_left, history, and is_recording are public instance variables and can be directly modified from outside the class. For a small project this is acceptable, but for better encapsulation, the class could provide getter methods or controlled update methods instead of allowing direct access to internal state.
	Is design by contract, meaning	Yes. There are some weak precondition

	pre/post-condition, violated?	and postcondition issues. execute_move() has a clear postcondition when a move succeeds: the starting cell becomes empty, the ending cell gets a peg, the jumped peg is removed, and pegs_left decreases by one. However, set_board_parameters() validates the board size but does not validate whether board_type is one of the allowed values. If an invalid board type is passed, the method still returns True and later _is_valid_slot() returns False for all positions, causing an invalid board state. Also, get_game_rating() assumes the game is already over, but this precondition is not checked inside the method.
	Is the Open-Closed Principle violated?	Yes, partially. Adding a new board type would require modifying _is_valid_slot() because the method uses an if/elif chain for "English", "Diamond", and "Hexagon". Adding a new board type would also require changing the GUI radio buttons. This means the design is not fully closed for modification. A better design would use separate board-shape classes or a board-shape strategy object so that new board types could be added without changing the existing validation method.
	Is the Single Responsibility Principle violated?	Yes, in some places. SolitaireGame handles board generation, move validation, move execution, rating calculation, recording setup, and saving record files. These responsibilities are related but could still be separated more clearly. For example, recording could be moved into a separate GameRecorder class. SolitaireGUI also has too many responsibilities: it builds the interface, handles user input, controls game flow, performs replay file parsing, animates replay, and handles save dialogs. Splitting replay and recording

		responsibilities into separate helper classes would improve the design.
Code Smells	Are there magic numbers?	Yes. The code uses several magic numbers. In the logic file, 0, 1, and 2 are used to represent board cell states. In the GUI file, numbers such as 700, 550, 420, 30, 0.3, and 0.4 are used for window size, canvas size, offsets, and replay delays. These should be replaced with named constants such as WINDOW_WIDTH, CANVAS_SIZE, BOARD_OFFSET, REPLAY_DELAY, INVALID_SLOT, PEG, and EMPTY.
	Are there unnecessary global / class variables?	No unnecessary global variables were found. Most state variables are instance variables, which is appropriate for this object-oriented design. However, there is one unnecessary import: os is imported in the test file but never used. Also, some instance variables such as history and is_recording are part of the recording feature, but they make the base game class responsible for recording as well as game logic.
	Is there duplicate code?	Some duplicate or similar logic exists. In SolitaireGUI, both on_new_game_click() and on_replay_click() contain logic for choosing between ManualGame and AutomatedGame, setting board parameters, and starting a new game. The coordinate calculation using offset, cell_size, row, and column also appears in both draw_board() and on_canvas_click(). These could be extracted into helper methods such as create_game_by_mode() and get_cell_from_click().
	Are there long methods?	Yes. The clearest example is SolitaireGUI.on_replay_click(), which is long and handles too many tasks at once: selecting a file, reading the file, validating the format, creating the correct game mode, updating GUI

		controls, applying moves, applying randomization, animating replay, and handling exceptions. _setup_ui() is also fairly long because it creates all GUI widgets in one method. These methods should be divided into smaller methods with one clear purpose each.
	Is there any long parameter list?	No. There is no extremely long parameter list, but the movement methods use four coordinate parameters: validate_move(sr, sc, er, ec) and execute_move(sr, sc, er, ec, is_replay=False). This is understandable for a board game, but readability could be improved by using tuples such as start_pos and end_pos, or a small Move object containing start row, start column, end row, and end column.
	Is there over-complex expression?	Some expressions are more complex than necessary. For example, validate_move() checks horizontal, vertical, and diagonal movement in one long condition. _is_valid_slot() also contains board-shape logic inside conditional expressions. These are not severe problems, but the code would be easier to maintain if valid movement directions were stored as named constants and board-shape logic were moved into separate helper methods or classes.
	Is there switch or if-then-else that needs to be replaced with polymorphism?	Yes. The if/elif structure in _is_valid_slot() checks the board type and applies different board-shape rules. This is a good candidate for polymorphism or the Strategy pattern. For example, EnglishBoard, DiamondBoard, and HexagonBoard could each implement an is_valid_slot() method. The current manual/automated game mode design already uses inheritance well, but the board type design still relies on conditionals.
	Any variable or method name whose	No, but some names could be clearer.

	intent is unclear?	The coordinate names sr, sc, er, and ec are short and may not be immediately understandable. The variable val in draw_board() is also vague because it represents the cell state. More descriptive names such as start_row, start_col, end_row, end_col, and cell_state would improve readability.	
	Any similar methods in different classes?	No major duplicated methods were found between ManualGame and AutomatedGame. The class hierarchy is mostly effective because common game logic is placed in SolitaireGame, while manual-only behavior such as randomize_board() is in ManualGame, and automated behavior such as make_automated_move() is in AutomatedGame. However, similar setup logic appears in different GUI methods, especially when creating a game object based on mode.	
Bugs	**Buggy code snippet**	**What is the bug?**	**Why is it a bug?**
	In SolitaireGUI.on_new_game_click(), the code creates a new ManualGame() or AutomatedGame() before validating the new board size. If the size is invalid, the GUI resets the entry to self.logic.board_size, which is the default value of the newly created game object.	Invalid board-size input can lose the previous valid board size. For example, if the current game size is 9 and the user enters -5, the GUI creates a new game object first. When validation fails, the size field is reset to the new object's default value, 7, instead of keeping the previous valid size 9.	This violates the intended behavior from the acceptance criteria that invalid board-size input should not replace the previous valid size. The logic class itself preserves the previous size when validation fails, but the GUI breaks this behavior by replacing the whole logic object before validation. A better design is to validate the input before creating a new game object, or store the previous valid size

			before replacing self.logic.
	In SolitaireGame.set_board_parameters(), the code validates size_str but does not validate board_type.	The method can accept an invalid board type, such as "Circle" or "WrongType", and still return True.	This can create an invalid game state. _is_valid_slot() only handles "English", "Diamond", and "Hexagon"; otherwise it returns False for all slots. As a result, a bad board type can lead to a board with no valid slots except the center being set to empty, and the game may start incorrectly. The method should reject board types outside the allowed set.
	In SolitaireGame.get_game_rating(), the method returns a rating based only on pegs_left without checking whether the game is actually over.	The method can return "Average", "Good", "Very Good", or "Outstanding" even when legal moves still exist.	According to the project description, the rating should be given after the game ends, when no more valid moves are possible or only one marble remains. In the current GUI, check_game_over() calls this method only after confirming no valid moves, so the GUI flow is mostly safe. However, the method itself does not enforce its precondition, which could cause incorrect rating

			results if another part of the program calls it too early.

2. Review how adequately you have tested your code and discuss how could you improve your tests based on lessons learned in the class (8 points).

   Do not simply answer “yes” or “no”. Give examples or the process you followed for “yes” answers. Concurrently, for “no” answers, discuss reasons and possible improvement.

   a. Have you performed acceptance tests for all user stories?

   I have performed acceptance tests for some user stories, but not for all of them completely. The current tests directly correspond to several important user stories from earlier sprints, such as choosing the board size, generating the initial board, making a manual move, performing an automated move, and randomizing the board state. For example, test_ac_1_1_valid_board_size tests whether the board size is updated to 9 after a valid size input. test_ac_1_2_invalid_board_size tests whether the game rejects an invalid size such as -5 and keeps the previous valid board size. test_ac_3_1_valid_jump tests whether a legal jump correctly changes the start cell to empty, places a peg in the destination cell, and decreases the total peg count by one. These tests are clearly related to the acceptance criteria for board setup, game initialization, and move execution.

   However, the acceptance tests are not complete for all user stories. In particular, the Sprint 4 record and replay features are not fully tested. The project requirements include recording a complete manual game, recording a manual game with randomization, recording an automated game, replaying a saved game from a text file, and restoring the board size and board type during replay. The current test_recording_feature only checks whether INIT, MOVE, and RANDOMIZE entries are added to the in-memory history list. It does not fully test whether save_record() writes the history to a file correctly, nor does it test whether the replay feature can read a saved file and reconstruct the correct game state. Therefore, the current acceptance tests are only partially adequate. They cover several major game logic features, but the record/replay user stories still need more direct and complete acceptance tests.

   b. Have you considered positive, negative and malicious scenarios in the acceptance tests?

   The current tests consider some positive and negative scenarios, but they do not sufficiently cover malicious scenarios. For positive scenarios, the test suite includes valid board size input, initial board generation, legal move execution, automated move execution, and automated game completion. For example, test_automated_move_execution first checks that there is at least one valid move, then calls make_automated_move(), and finally verifies that the number of pegs decreases by one. This is a typical positive test case because it verifies the expected behavior under normal conditions. For negative scenarios, the test suite includes invalid board size input and invalid out-of-bounds movement. For example, test_ac_1_2_invalid_board_size uses -5 as an invalid input and verifies that the game rejects it.

test_ac_3_4_invalid_out_of_bounds attempts to move a peg outside the board and verifies that the move returns False. These tests reflect the idea of testing invalid or failure equivalence classes, which was discussed in Chapter 8.

However, malicious or extreme scenarios are not adequately covered. For example, the current tests do not check board size inputs such as "abc", "7.5", an empty string, or a very large number. They also do not test invalid board types such as "Circle" or "WrongType". For the record/replay feature, the tests do not check corrupted record files, missing INIT lines, incomplete MOVE lines, invalid JSON in RANDOMIZE lines, or unwritable file paths. Based on the testing concepts from Chapter 8, black-box testing should be designed from the specification, and dirty or invalid equivalence classes should be considered carefully. From that perspective, the current tests cover some ordinary invalid cases, but they do not yet sufficiently test malicious inputs or abnormal file-handling situations.

c. Have you written unit tests (beyond the acceptance tests) for the implemented features?

Yes, I have written some unit tests beyond direct acceptance tests, but the number of such tests is still limited. For example, test_randomize_board does more than simply check whether the randomization method can be called. It checks that the number of pegs before and after randomization remains the same. It also counts the actual number of pegs in the grid and compares it with pegs_left. This verifies an important invariant: the internal peg counter must match the actual board state. Another example is test_automated_game_finishes. This test repeatedly calls make_automated_move() until no valid moves remain, then verifies that has_any_valid_moves() returns False. This is not just a one-step acceptance test. It checks the behavior of the automated game over a sequence of moves.

However, more unit tests are still needed. For example, the current final test file does not separately test all branches of get_game_rating(). It also does not directly test save_record(), the replay branch of randomize_board(is_replay=True, grid_state=...), or all board-shape branches in _is_valid_slot() for English, Diamond, and Hexagon boards. According to the white-box testing concepts in Chapter 8, tests should try to cover statements, branches, conditions, and implemented behaviors as much as possible. Based on this standard, the current unit tests should be expanded.

d. What is the code coverage obtained for testing the main features?

After running the current tests with a coverage tool, the statement coverage for logic.py is about 85%. This means that most of the core game logic has been executed by the test suite, including board parameter setup, game initialization, move validation, move execution, randomization, and automated movement. The current test suite runs 9 tests successfully, which shows that the existing logic-level tests are repeatable and stable.

However, if gui_app.py is included in the overall project coverage measurement, the total coverage drops to about 47%. This is because the current automated tests mainly target the logic layer and do not execute most GUI methods. In other words, the test coverage is relatively good for the game logic, but weak for the complete system. This result matches the coverage concept discussed in Chapter 8: coverage can help measure whether tests execute methods, statements, branches, paths, and conditions. However, coverage alone does not

prove that all behavior is correct. It only shows which parts of the code were executed. Since the GUI is an important part of the final product, the current tests are acceptable for the main game logic but not sufficient for the whole application.

e. Have you added tests to improve code coverage?

Yes, I have added tests across the sprints to improve coverage. Earlier tests mainly focused on board size, board type, board initialization, movement, and rating logic. Later tests added coverage for manual mode, automated mode, board randomization, and recording behavior. The current test file includes test_recording_feature, which checks whether the game records initialization, movement, and randomization actions in the history list.

However, more tests should be added to further improve coverage. First, I should test all rating outcomes in get_game_rating(): one peg should return "Outstanding", two pegs should return "Very Good", three pegs should return "Good", and four or more pegs should return "Average". Second, I should test board initialization for English, Diamond, and Hexagon boards to verify that the center is empty, valid slots contain pegs, and invalid slots remain zero. Third, I should test whether save_record() actually writes the history to a text file and whether the file content can be read back correctly. Fourth, I should test replay-related behavior, especially whether a recorded RANDOMIZE grid can be restored correctly. Finally, I should add tests for GUI methods such as on_new_game_click(), on_randomize_click(), on_autoplay_click(), on_replay_click(), and check_game_over(), because those methods currently contain important behavior that is mostly untested.

f. Have you performed any other automated tests e.g., fuzzing?

No, I have not yet performed fuzzing or other advanced automated testing techniques. The current tests are mainly handwritten unittest cases with fixed inputs. They are automated in execution, but the test inputs themselves are manually designed.

Chapter 8 discusses automated test generation, including random, biased-random, and directed approaches. Based on that idea, fuzzing could be useful for this project. For example, I could randomly generate board size inputs, including negative numbers, even numbers, odd numbers, strings, empty strings, and very large numbers, then test whether set_board_parameters() always returns True or False safely without crashing.

I could also randomly generate coordinate tuples such as (sr, sc, er, ec) and repeatedly call validate_move() or execute_move() to make sure the program does not crash with an index error and that invalid moves do not change the board state. For the record/replay feature, I could generate malformed record lines such as incomplete MOVE lines or invalid RANDOMIZE JSON strings to check whether replay fails safely instead of crashing.

These fuzzing-style tests would be consistent with fault-based testing. Instead of only testing expected behavior, they would intentionally search for runtime errors, boundary problems, invalid input handling issues, and file parsing failures.

g. How do you improve your current testing of the code, especially the testing of the game logic?

First, for black-box testing, I would create a traceability matrix that maps every user story and acceptance criterion to at least one test case. Chapter 8 emphasizes that black-box

tests should be based on the specification and should be traceable to requirements or acceptance criteria. Therefore, I would add more tests for the Sprint 4 record/replay requirements. For AC 20.1, I would test whether both manual and automated games can record INIT and MOVE entries when recording is enabled. For AC 20.2, I would test whether randomization records the full grid state. For AC 21.1 and AC 21.2, I would create temporary record files and verify that replay restores the correct board size, board type, game mode, and final board state.

Second, I would apply equivalence partitioning and boundary value analysis more systematically. The current test suite only tests a valid size such as "9" and an invalid size such as "-5". A stronger design would divide board size inputs into valid odd numbers, invalid even numbers, invalid non-numeric inputs, and boundary values. For example, "3" should be tested as the smallest valid board size, while "1" should be tested as an invalid value below the minimum. Inputs such as "4", "abc", "", and "7.5" should also be tested. The same idea can be applied to board types and move coordinates.

Third, for white-box testing, I would add tests according to the actual branches in the code. validate_move() has several different ways to return False: out-of-bounds coordinates, a starting cell without a peg, a destination cell that is not empty, an invalid movement distance, or an empty middle cell. The current tests only cover a valid move and one out-of-bounds case. More tests should be added for the other invalid move paths. In addition, _is_valid_slot() contains different branches for English, Diamond, and Hexagon boards, but the current tests mainly use the English board. To improve branch coverage, I would add tests for all three board types. Overall, the current test suite verifies many important features of the Solitaire game logic, including board initialization, basic movement, randomization, recording history, and automated movement. However, it still needs improvement. The main weaknesses are the incomplete testing of replay, the lack of GUI integration tests, insufficient testing of Diamond and Hexagon boards, limited use of equivalence partitioning and boundary value analysis, and the absence of fuzzing or other fault-based tests. In the future, I would improve the test suite by mapping every acceptance criterion to a test case, increasing branch coverage, adding file-based record/replay tests, and using randomized testing to check robustness.

3. Summarize the lessons learned from the entire project by answering the following questions from the perspectives of development processes, coding, design, refactoring, and testing (4 points):

• What did you personally gain from the project?

Through the entire Solitaire project, I gained a much deeper understanding of how software engineering is different from simply writing code that works once. At the beginning of the project, my work was mostly focused on making basic technical decisions and building a small prototype. In Sprint 0, I chose Python as the programming language, Tkinter as the GUI library, PyCharm as the IDE, unittest as the testing framework, Google Python Style Guide as the coding style, and GitHub as the project hosting site. At that stage, the project was still very small. The logic class only stored a board size and a peg count, and the tests only checked the initial peg count and whether a jump distance was valid. However, even that simple start helped me understand that early decisions about tools, style, testing framework,

and project structure affect the rest of the development process.

As the project continued, I learned how requirements gradually become a working product. In Sprint 1, I had to write user stories and acceptance criteria from the player's point of view. This was useful because it forced me to think about what the user actually wants, not just what methods I need to write. For example, the early user stories included choosing a board size, choosing a board type, starting a new game, making a move, and detecting when the game is over. The acceptance criteria made these requirements more concrete by describing expected behaviors such as accepting valid board sizes, rejecting invalid inputs, keeping the previous valid size, rejecting invalid moves, and displaying the correct game rating. I also learned that requirements must be precise. When an LLM generated vague or unrealistic criteria, I had to correct them based on the actual project context, such as the fact that the game already has a default board size and a default English board type. This taught me that AI tools can support development, but the developer is still responsible for checking whether the generated content is correct and relevant.

From the coding perspective, I gained experience in turning a set of user requirements into actual classes, methods, and tests. In Sprint 2, I implemented the core game logic, including board setup, board generation, move validation, move execution, game-over checking, and rating calculation. This helped me understand the importance of separating the game logic from the GUI. If the logic is separated from the interface, it can be tested more easily and reused more safely. I also learned that a test should check all important results of an operation. For example, after a valid move, it is not enough to check only that the grid changes; the test should also check that the total peg count decreases by one. This improved my understanding of test oracles and state-based testing.

From the design and refactoring perspective, Sprint 3 was the most valuable part of the project. The project had to support both manual and automated game modes, so I refactored the design into a class hierarchy. The SolitaireGame base class became responsible for shared game rules and state, while ManualGame handled manual-specific behavior such as randomization, and AutomatedGame handled automatic move selection. This helped me better understand inheritance, polymorphism, cohesion, coupling, and encapsulation. Before refactoring, the design could easily become too monolithic because one class might try to handle board geometry, user input, randomization, and automated decision-making all at once. After refactoring, the responsibilities became clearer, and the GUI could interact with a general game object without needing to know every implementation detail.

From the testing perspective, I learned that tests should grow together with the project. Early tests were simple, but later tests needed to cover more complex behavior such as valid moves, invalid moves, randomization, automated play, recording, and replay. I also learned that testing is not only about proving that the normal path works. Good tests should also check boundary cases, invalid inputs, failure cases, and internal consistency. For example, after randomizing the board, the number of actual pegs in the grid should still match the pegs_left variable. Overall, I personally gained a more complete view of software development as an iterative process involving requirements, design, coding, testing, refactoring, and review.

• What does your project do well, and what could your project do better?

The project does several things well. First, it implements the major required features of

the Solitaire game. The final version supports choosing a board size and board type, starting a new game, making legal moves, rejecting invalid moves, detecting when the game is over, and assigning ratings based on the number of pegs left. It also supports manual and automated modes. In automated mode, the computer selects valid moves instead of using a hardcoded move sequence. This makes the automated mode more flexible and more consistent with the project requirements.

Second, the project has a reasonably good separation between the game logic and the GUI. The logic classes are responsible for board state, movement rules, peg counting, game-over checking, randomization, recording, and automated movement. The GUI is responsible for displaying the board and responding to user actions. This separation makes the logic easier to test with unittest. It also makes the code easier to understand because the rules of the game are not completely mixed with Tkinter event-handling code.

Third, the project uses object-oriented design in a meaningful way. The class hierarchy with SolitaireGame, ManualGame, and AutomatedGame helps avoid duplicated code. Shared behavior such as board generation, move validation, move execution, and valid move searching is placed in the base class. Mode-specific behavior is placed in the subclasses. This improves modularity and cohesion. It also reduces coupling because the GUI can work with either a manual or automated game object through a shared interface.

Fourth, the project was improved through incremental development. Each sprint added a new layer of functionality. Sprint 0 established the basic tools and prototype. Sprint 1 clarified user stories and acceptance criteria. Sprint 2 implemented the basic game features and unit tests. Sprint 3 added manual and automated game modes and refactored the design. Sprint 4 added recording and replay. Sprint 5 then provided an opportunity to review the code, review the tests, and reflect on the whole process. This incremental process made the project more manageable than trying to build the entire application at once.

However, the project could still be improved in several ways. One weakness is that the board state is represented using raw integer values such as 0, 1, and 2. Although the code works, these values are not self-explanatory. It would be better to define named constants such as INVALID_SLOT, PEG, and EMPTY. This would make the code easier to read and reduce the chance of misunderstanding the meaning of each value.

Another area for improvement is the board type design. Currently, the board shape logic is handled with conditional statements for English, Diamond, and Hexagon boards. If I wanted to add another board shape in the future, I would need to modify the existing validation logic. This partially violates the Open-Closed Principle. A better design would use separate board shape classes or a strategy pattern, so each board type could define its own valid-slot rule without changing the core game class.

The GUI code could also be improved. Some GUI methods handle too many responsibilities at once. For example, replay-related logic involves choosing a file, reading the file, parsing text, creating the correct game object, updating the GUI, applying moves, handling randomization, and animating the replay. This works, but it makes the method long and harder to test. A better approach would be to split these tasks into smaller helper methods or separate classes, such as a record parser or replay controller.

The test suite is useful but not complete. The current tests cover many important logic-level features, including valid and invalid board sizes, initial board state, valid jumps, out-of-

bounds moves, randomization, recording history, automated move execution, and automated game completion. However, the project could do better by testing the replay feature more directly, testing file writing and file parsing, testing invalid record files, testing GUI interactions with mocks, and testing all board types more thoroughly. In short, the project works as a functional application, but it could be more maintainable, extensible, and thoroughly tested.

• How could you improve your development process if you develop a similar game from scratch?

If I developed a similar game from scratch, I would improve my development process in several ways. First, I would spend more time designing the domain model before writing the GUI. In this project, the design improved over time, but some concepts such as board states, board shapes, moves, and recording could have been modeled more clearly from the beginning. For a new project, I would define these concepts as separate responsibilities early. For example, I would consider creating a Move object to represent start and end coordinates, a Board or BoardShape abstraction to handle board geometry, and a GameRecorder or ReplayManager to handle file-based recording and replay. This would make the design cleaner and reduce the need for later refactoring.

Second, I would connect user stories, acceptance criteria, and tests more directly from the beginning. In this project, the acceptance criteria were useful, but the tests did not always cover every criterion completely. For a future project, I would create a traceability table where each user story and acceptance criterion has at least one corresponding test case. This would make it easier to know which requirements are fully tested and which ones still need work. I would also write the acceptance criteria in a more testable way from the start, using clear inputs, actions, and expected outcomes.

Third, I would use test-driven development more consistently for the core game logic. The movement rules and game-over detection are the most important parts of the Solitaire game, and they are also the easiest parts to get subtly wrong. If I were starting over, I would write tests for legal moves, illegal moves, boundary positions, empty middle cells, occupied destination cells, and different board shapes before or alongside implementation. This would help catch errors earlier and make refactoring safer.

Fourth, I would keep the GUI thinner. In the current project, the GUI does not only display the board; it also contains important control flow for creating games, starting automated play, saving records, and replaying files. In a future version, I would move more of this behavior into separate controller or service classes. The GUI should mainly collect user input and display results, while the logic and controller layers should handle rules, state transitions, file parsing, and replay steps. This would make the system easier to test because GUI event handlers are harder to test than ordinary methods.

Finally, I would use GitHub more deliberately as part of the development process. Instead of making large changes at once, I would make smaller commits with clearer messages, such as “add board size validation,” “implement valid move detection,” “refactor manual and automated game hierarchy,” or “add replay file parser.” This would make the project history easier to understand and make it easier to return to a previous version if a bug appears.

Overall, if I developed a similar game again, I would start with a clearer architecture,

connect requirements to tests more tightly, write more tests earlier, keep the GUI less responsible for business logic, and refactor continuously. The biggest lesson from this project is that software quality is not something added only at the end. It is built gradually through good requirements, clean design, readable code, meaningful tests, and regular review.